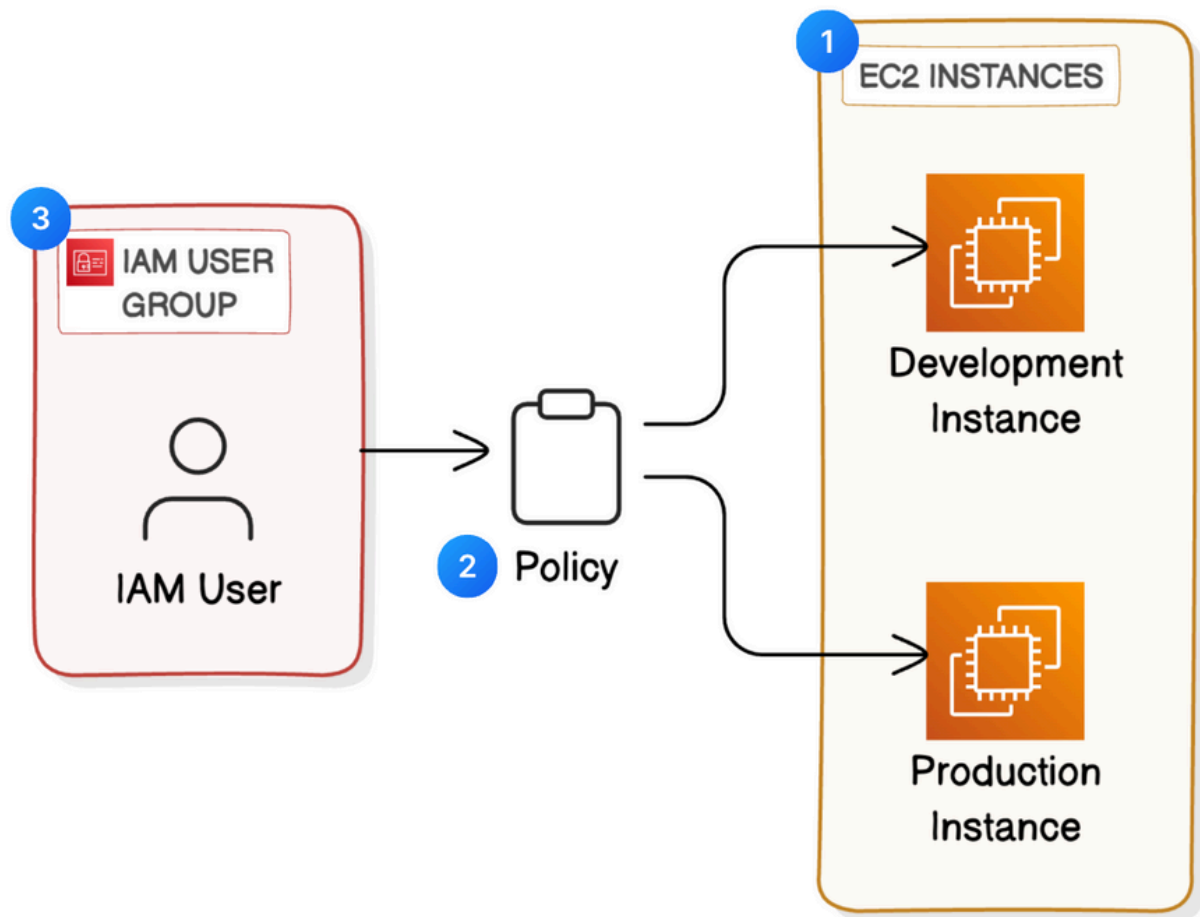


CLOUD SECURITY AWS IAM

Muhammad Anas Naeem | Jun 18, 2025

[LinkedIn](#)



Overview

As someone working in cybersecurity, I wanted to take my first real step into **cloud security**. To do this, I completed a practical **AWS IAM** project that helped me understand and implement real-world access control using Amazon Web Services.

The focus was on setting up **secure, tag-based access** to multiple **EC2** instances using custom **IAM policies**. It was a satisfying and insightful experience that gave me hands-on exposure to how permissions, policies, and identity management work together in the cloud.

What is AWS and IAM?

Amazon Web Services (AWS) is the world's leading cloud computing platform, used by startups and enterprises alike to host applications, store data, and run services securely at scale.

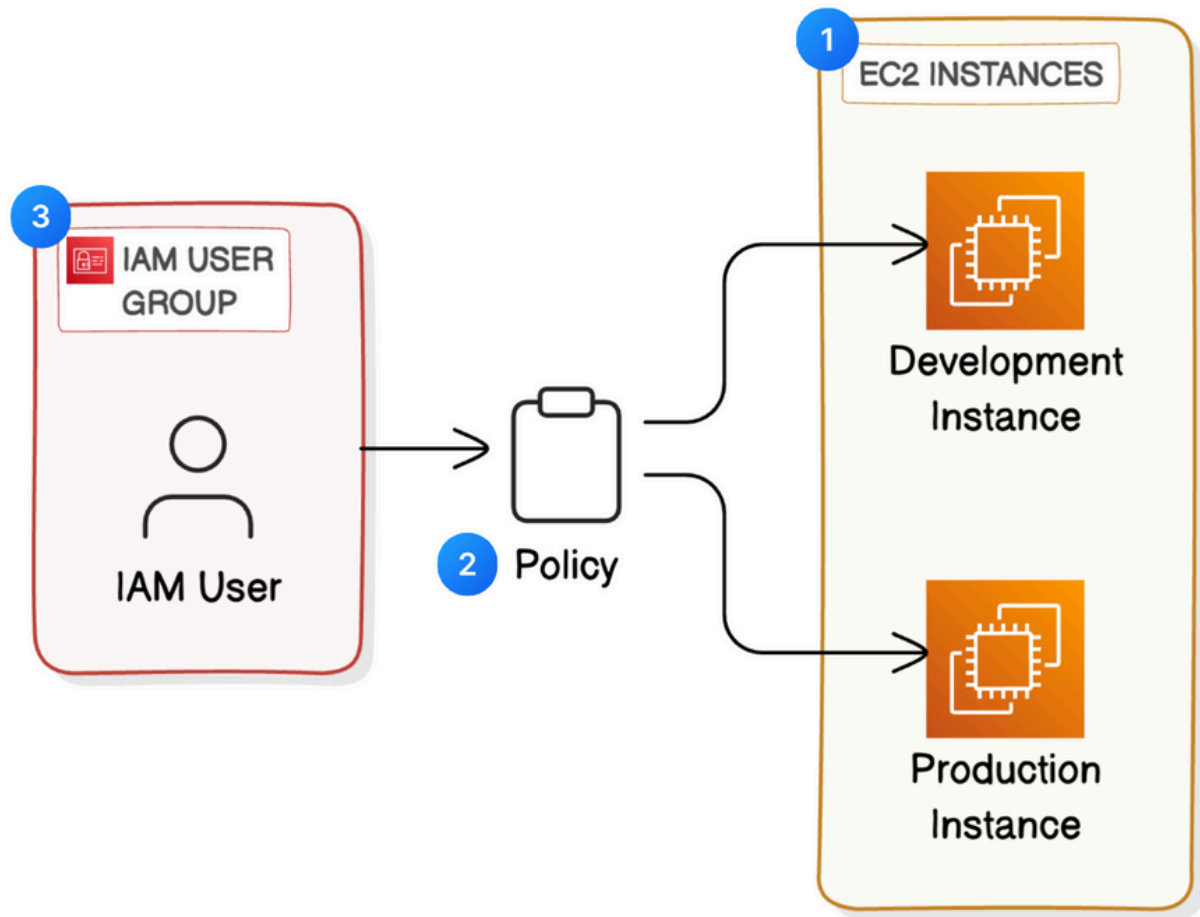
Identity and Access Management (IAM) is AWS's built-in service for controlling who can access what within your cloud environment. It allows you to create **users**, **groups**, and **policies** to manage permissions in a precise and secure way.

With IAM, you can ensure that people and systems only have access to the resources they need, nothing more, nothing less.

Objective

To simulate a real-world cloud security setup by

- Launching two EC2 instances — one for **development** and one for **production**
- Creating IAM users and groups
- Writing a **custom IAM policy** that restricts access to only the development instance
- Enforcing the **principle of least privilege**
- Testing access through both the AWS console and Policy Simulator



Project Architecture

IAM User → IAM Group → Custom IAM Policy → EC2 (Tag: Environment)
↳ Access to "development" only

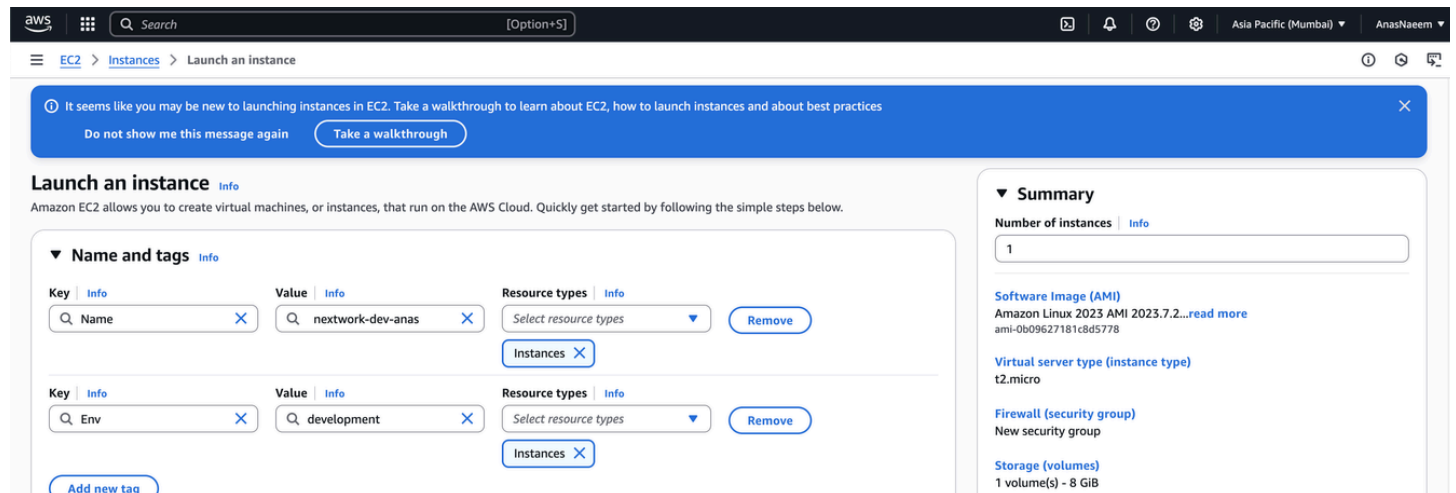
Tools & Services Used

Tools	Description
AWS IAM	Users, groups, policies
EC2	Instances with environment tagging
JSON IAM Policy Language	For custom access control
AWS Policy Simulator	For safe pre-deployment testing

What I Did

During this project, I followed a structured, step-by-step approach to build and test IAM-based access control:

I began by launching two **Amazon EC2 instances**, one tagged as a **development environment** and the other as **production**. This separation was important for simulating real-world cloud workflows.



The screenshot displays the AWS Management Console interface for launching an EC2 instance. The top navigation bar shows the AWS logo, a search bar, and the user's profile. The main content area is titled 'Launch an instance' and includes a blue banner with a walkthrough message. Below this, the 'Name and tags' section allows for adding tags. Two tags are currently added: 'Name' with the value 'nextwork-dev-anas' and 'Env' with the value 'development'. The 'Summary' section on the right provides a overview of the instance configuration, including the number of instances (1), the software image (Amazon Linux 2023 AMI), the virtual server type (t2.micro), the firewall (new security group), and the storage (1 volume of 8 GiB).

EC2 Instances Tagging

Next, I created an **IAM user** and added it to a **group** that would receive restricted access. With this setup, I then wrote a **custom IAM policy in JSON**, which allowed EC2 actions *only* on instances tagged with Environment: development. Access to the production instance was strictly denied.

Policy editor

Visual

JSON

Actions

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "ec2:*",
7       "Resource": "*",
8       "Condition": {
9         "StringEquals": {
10          "ec2:ResourceTag/Env": "development"
11        }
12      },
13    },
14    {
15      "Effect": "Allow",
16      "Action": "ec2:Describe*",
17      "Resource": "*"
18    },
19    {
20      "Effect": "Deny",
21      "Action": [
22        "ec2:DeleteTags",
23        "ec2:CreateTags"
24      ],
25      "Resource": "*"
26    }
27  ]
28 }

```

+ Add new statement

Edit statement

Select a statement

Select an existing statement in the policy or add a new statement.

+ Add new statement

JSON Ln 29, Col 0

5851 of 6144 characters remaining

Security: 0

Errors: 0

Warnings: 0

Suggestions: 0

Custom IAM Policy

IAM > Users > Create user

User created successfully

You can view and download the user's password and email instructions for signing in to the AWS Management Console.

View user

Step 1

Specify user details

Step 2

Set permissions

Step 3

Review and create

Step 4

Retrieve password

Retrieve password

You can view and download the user's password below or email users instructions for signing in to the AWS Management Console. This is the only time you can view and download this password.

Console sign-in details

Email sign-in instructions

Console sign-in URL

https://nextwork-alias-anas.signin.aws.amazon.com/console

User name

nextwork-dev-anas

Console password

***** Show

Cancel

Download .csv file

Return to users list

IAM User Created

To ensure everything worked as expected, I used the **AWS Policy Simulator** to verify that the policy behaved correctly, granting access to development and denying production. Finally, I logged in with the IAM user and manually confirmed that the permissions aligned with the policy rules in the live AWS Console.

Policy Simulator

Amazon EC2

1 Action(s) sele...

Select All

Deselect All

Reset Contexts

Clear Results

Run Simulation

Global Settings ⓘ

Action Settings and Results [2 actions selected. 0 actions not simulated. 1 actions allowed. 1 actions denied.]

Service	Action	Resource Type	Simulation Resource	Permission
Amazon API Gateway	Invoke	not required	*	denied Implicitly denied (no matching state...
Resource You can specify the resource and context keys used to simulate this action. By default the simulation resource is "".				
Add Resource				
Amazon EC2	DescribeHosts	not required	*	allowed 1 matching statements.
Show statement in NextWorkDevEnvironmentPolicy (IAM Policy)				
Resource This action does not support resource-level permissions. Policies granting access must specify "" in the resource element.				
ARN				

IAM Policy Simulator

Why Development Access

In secure cloud environments, **production systems must always be tightly controlled**. This project reflected that mindset — only the development instance was accessible to prevent accidental disruption or unauthorized changes in production.

This follows the **principle of least privilege**, ensuring users have access only to what they need.

Key Lessons Learned

Tag-based policies are a powerful way to manage access at scale

Testing IAM policies before rollout is essential to avoid misconfigurations

IAM permissions can be fine-tuned with precise, condition-based rules

Writing structured policies is not only critical, it's satisfying when done right

To conclude, this project marked an important milestone in my transition from traditional cybersecurity to the dynamic world of cloud security. By working hands-on with AWS IAM, I not only strengthened my understanding of identity and access control but also gained practical experience writing and testing real-world policies.

It reinforced the importance of least privilege, tagging strategies, and policy simulation, all critical skills for secure cloud environments. This experience has deepened my confidence and laid a strong foundation for more advanced AWS security projects in the near future.

Muhammad Anas Bin Naeem